

CAPÍTULO 5

Agent Skills: Criando a sua Matrix

Em 1999, uma cena mudou para sempre o imaginário de quem trabalha com tecnologia. Trinity, encurralada no topo de um prédio, precisa pilotar um helicóptero militar para resgatar Morpheus. Ela não sabe pilotar. Então liga para Tank, o operador da Nabucodonosor, e pede: 'Tank, I need a pilot program for a B-212 helicopter. Hurry.' Tank carrega o programa. Os olhos de Trinity se movem rápido por dois segundos. Ela abre os olhos e diz: 'Let's go.' Dois segundos. Do zero absoluto ao domínio completo de uma aeronave militar. Não foi estudo, não foi prática, não foi tentativa e erro. Foi uma **skill** injetada diretamente no sistema operacional de um agente inteligente.

Essa cena, que em 1999 era ficção científica pura, é a metáfora exata do que aconteceu com o desenvolvimento de software em 2025. Mas antes de avançar, vale fixar dois conceitos que sustentam todo o capítulo. Um **agente inteligente** é um programa que percebe o ambiente ao redor, decide o que fazer e age por conta própria. Não espera você digitar cada passo; ele analisa o contexto, escolhe a próxima ação e executa. Uma **skill** é a forma de transformar esse agente genérico em um colaborador especializado. Em vez de repetir instruções toda vez, você empacota contexto, regras e fluxo de trabalho em um artefato reutilizável que o agente carrega instantaneamente, do mesmo jeito que Tank carrega o programa de pilotagem na Trinity.

Quando a Anthropic lançou o sistema de **Agent Skills** no **Claude Code**, o que antes era um assistente que respondia perguntas virou algo fundamentalmente diferente: um agente que recebe habilidades modulares e executa tarefas complexas com autonomia. Em poucos meses, o padrão atravessou a fronteira da Anthropic. A OpenAI adotou exatamente o mesmo formato no **Codex**, e o Google fez o mesmo no **Antigravity**. As três ferramentas hoje convergem em torno do mesmo arquivo, **SKILL.md**, com a mesma anatomia. O que muda entre elas são detalhes de onde os arquivos moram e como o agente é configurado, não o conceito. O programador deixou de ser o digitador e passou a ser o operador da Nabucodonosor,

orquestrando agentes que possuem habilidades que ele mesmo definiu, em qualquer das três ferramentas. A Figura 5.1 resume essa metáfora central.

Agent Skills A SUA MATRIX

O programador se torna o operador
que define, injeta e orquestra
habilidades em agentes inteligentes



Figura 5.1: Assim como Trinity recebe habilidades instantâneas na Matrix, os agentes inteligentes modernos recebem Agent Skills que transformam suas capacidades em tempo real. O programador se torna o operador que define, injeta e orquestra essas habilidades.

O que torna esse momento genuinamente novo não é apenas a capacidade de executar código. Chatbots já faziam isso. O que muda é a arquitetura ao redor da execução. Uma **skill** não é um prompt salvo; é um módulo com contexto, regras de ativação e comportamento reutilizável que o agente carrega quando precisa, sem que ninguém precise pedir. Um **hook** é um sentinela que intercepta cada ação do agente antes ou depois da execução, decidindo se permite, bloqueia ou modifica o que está acontecendo. E o ambiente isolado do **.venv** garante que cada skill opere com suas próprias dependências, sem contaminar o restante do sistema. Juntas, essas peças formam um ecossistema que dá ao agente consciência do ambiente ao redor.

A analogia com a Matrix não é decorativa. Ela é estrutural. Cada conceito deste capítulo tem um paralelo direto no universo criado pelos Wachowskis, e essa correspondência não é forçada; ela emerge naturalmente porque ambos os sistemas resolvem o mesmo problema fundamental: como dar a um agente capacidades que ele não nasceu tendo, em um ambiente que precisa ser controlado, monitorado e orquestrado por alguém que entende o quadro completo. Na Matrix, esse alguém é o operador. No desenvolvimento moderno de software, esse alguém é você.

Ao longo deste capítulo, três ferramentas servem de referência concreta: **Claude Code** (Anthropic), **Codex** (OpenAI) e **Antigravity** (Google). Sempre que a configuração for idêntica nas três, ela aparece uma única vez. Sempre que houver diferença real — caminho de arquivo, formato do manifesto de configuração, comando de invocação —, as três versões aparecem lado a lado, claramente rotuladas. O leitor escolhe a sua ferramenta e o livro continua sendo

referência fiel para o que ele tem em mãos. Começamos distinguindo **skills** do **MCP**, o protocolo que conecta o agente a ferramentas externas e que é tratado em profundidade no livro dedicado do autor¹. Em seguida, mergulhamos na anatomia das skills: o que são, como funcionam por dentro e por que são diferentes de um simples prompt. Depois, respondemos uma pergunta que parece óbvia mas raramente é bem explicada: o que exatamente é um **agente inteligente**, qual é o ciclo que o separa de um chatbot e por que isso importa para quem escreve software. Entramos então no território dos **hooks**, os sentinelas que monitoram e controlam cada ação do agente. Passamos pelo **.venv**, o ambiente isolado que garante que cada skill tenha suas próprias dependências sem contaminar o sistema. E fechamos com uma missão prática: você vai criar uma skill que estrutura um projeto Python, prepara o ambiente virtual e roda um pequeno script que valida toda a cadeia de ponta a ponta.

Tank não carregou meia habilidade na Trinity. Carregou o programa completo. Este capítulo faz o mesmo.

5.1 TRINITY E O CHAVEIRO: DOIS NÍVEIS DE PODER

Na trilogia Matrix, dois personagens representam dois tipos de poder completamente distintos. Trinity recebe habilidades: pilotagem de helicóptero, combate corpo a corpo, operação de armas. Cada habilidade é carregada diretamente nela e passa a fazer parte do que ela é capaz de fazer. Ela não precisa de ninguém depois que a habilidade é instalada. É poder **interno**. O Chaveiro, por outro lado, não tem nenhuma habilidade de combate. O que ele tem são chaves. Chaves que abrem portas para lugares que ninguém mais consegue acessar: a sala do Merovíngio, os corredores entre mundos, a fonte da Matrix. Ele não faz nada sozinho, mas conecta quem precisa ao lugar certo. É poder **externo**.

Essa distinção é exatamente a que existe entre **Agent Skills** e **MCP** nos três ecossistemas que o livro toma como referência: **Claude Code**, **Codex** e **Antigravity**. Os três adotaram o mesmo modelo de dois pilares, e entender a diferença entre eles é o primeiro passo para operar a Matrix com competência — em qualquer uma das três.

Uma **Agent Skill** é uma habilidade injetada no agente. Quando você cria uma skill que ensina o agente a revisar código seguindo um padrão específico, a gerar imagens no estilo do seu projeto, ou a executar um deploy com um único comando, você está fazendo o que Tank faz na Nabucodonosor: carregando um programa diretamente no operacional do agente. A partir daquele momento, o agente sabe fazer aquilo. A skill fica dentro dele, faz parte do seu repertório, e pode ser ativada por você ou pelo próprio agente quando ele julgar necessário. Não depende de serviço externo, não precisa de conexão com nada fora do ambiente local. É poder interno, como Trinity depois de receber o programa de pilotagem.

¹ Para MCP: <http://physia.com.br/mcp>

O **MCP**, Model Context Protocol, é o Chaveiro. Ele não dá habilidades ao agente; ele abre portas para ferramentas e serviços externos. Um servidor MCP conecta o agente ao GitHub, ao banco de dados, ao Slack, ao Firebase, ao Google Calendar. O agente passa a poder acessar esses lugares, ler dados, executar ações, mas a conexão não é dele; é do protocolo que abre a porta. Se o servidor MCP for desligado, a porta fecha. O agente não perdeu nenhuma habilidade interna; perdeu acesso a um lugar externo. Trinity sem o programa de pilotagem não sabe pilotar. O Chaveiro sem as chaves ainda é o Chaveiro, mas não abre nenhuma porta.

Os dois são complementares, não concorrentes. Um agente verdadeiramente poderoso tem ambos: skills que definem o que ele sabe fazer e conexões MCP que definem onde ele pode agir. Este capítulo é inteiramente dedicado ao primeiro pilar: as **Agent Skills**. O **MCP** tem tratamento completo no livro dedicado do autor², e será referenciado sempre que a conexão entre os dois pilares for relevante.

Para navegar o restante deste capítulo, a tabela 5.1 serve como mapa. Cada conceito técnico que você vai encontrar nas próximas seções tem um paralelo direto na Matrix, e essas analogias não são ilustrativas; elas capturam a essência funcional de cada componente. Os mecanismos listados existem nas três ferramentas que servem de referência ao livro — **Claude Code**, **Codex** e **Antigravity** — com o mesmo papel funcional, ainda que os nomes e caminhos exatos variem em pontos específicos detalhados nas próximas seções.

Guarde essa tabela. Ela é o índice conceitual de tudo que vem pela frente. A partir da próxima seção, cada linha desta tabela ganha profundidade técnica, código real e exemplos práticos. Começamos pelo mais fundamental: o que exatamente é uma Agent Skill e como ela funciona por dentro.

5.2 BAIXANDO HABILIDADES: O QUE SÃO AGENT SKILLS?

Quando Tank carrega o programa de pilotagem na Trinity, ele não entrega um manual de instruções. Ele injeta um módulo completo: o que fazer, quando ativar, quais instrumentos usar, quais limites respeitar. A Trinity não precisa interpretar nada. Ela abre os olhos e sabe pilotar. Uma **Agent Skill** funciona exatamente assim. Não é um texto solto que o agente lê e tenta seguir. É um módulo estruturado com metadados, condições de ativação, ferramentas pré-aprovadas, regras de execução e formato de saída. É a diferença entre entregar um livro de receitas para alguém que nunca cozinhou e instalar um chef completo dentro da pessoa.

² Para MCP: <http://physia.com.br/mcp>

Matrix	Mecanismo nos Agentes	Por que funciona
Trinity baixando pilotagem	Agent Skills	Habilidade instantânea injetada no agente
O Chaveiro	MCP	Abre portas para ferramentas e serviços externos
Agent Smith	Hooks	Intercepta ações antes ou depois; sempre de olho
A tripulação da Nabucodonosor	Subagents	Cada membro com seu papel na missão
Os clones do Agent Smith	Subagents em paralelo	Múltiplas instâncias rodando ao mesmo tempo
O Construct	Worktrees	Espaço isolado para trabalhar sem afetar o mundo real
O Operador (Tank)	Workflows	Orquestra a missão de fora, coordenando tudo
A ligação telefônica	Slash Commands	O gatilho que aciona a habilidade
Os programas do Merovíngio	Plugins	Pacotes externos com poderes específicos

Tabela 5.1: Mapa de analogias: a Matrix e o ecossistema de Agent Skills, válido para Claude Code, Codex e Antigravity.

Onde moram as habilidades

Toda **skill**, nas três ferramentas, é um diretório que contém um arquivo chamado `SKILL.md` — o cérebro da habilidade — e opcionalmente arquivos de suporte: scripts, templates, exemplos, referências. O conceito é idêntico. O que muda entre Claude Code, Codex e Antigravity é apenas o caminho onde esse diretório vive. Cada ferramenta procura skills em locais próprios, e o leitor precisa saber exatamente onde gravar o arquivo para que o agente a enxergue.

No **Claude Code**, o diretório de skills do projeto é `.claude/skills/`, na raiz do repositório. Skills disponíveis em todos os projetos do usuário ficam em `~/.claude/skills/`.

```
# Claude Code
.claude/
  skills/
    minha-skill/
      SKILL.md           # o cerebro da habilidade
      scripts/           # scripts de apoio (opcional)
      references/        # docs internos (opcional)
```

No **Codex** (OpenAI), o projeto pode usar dois caminhos equivalentes para o mesmo fim: `.agents/skills/` (formato compartilhado com o Antigravity) ou `.codex/skills/` (ca-

minho específico do Codex). O Codex percorre essas pastas a partir do diretório de trabalho subindo até a raiz do repositório. Skills do usuário ficam em ~/.codex/skills/ ou ~/.agents/skills/, e ainda há um nível de sistema em /etc/codex/skills/ para máquinas administradas.

```
# Codex (OpenAI) - duas pastas validas, escolha uma
.agents/skills/minha-skill/SKILL.md
# ou
.codex/skills/minha-skill/SKILL.md
```

No **Antigravity** (Google), o caminho padrão também é .agents/skills/ no *workspace*, com retrocompatibilidade para .agent/skills/ (no singular, sem o s). Skills globais ficam em ~/.gemini/antigravity/skills/.

```
# Antigravity (Google)
.agents/
  skills/
    minha-skill/
      SKILL.md           # mesma anatomia novamente
      scripts/
      references/
```

A tabela 5.2 resume os caminhos para consulta rápida.

Ferramenta	Skills do projeto	Skills do usuário
Claude Code	.claude/skills/	~/.claude/skills/
Codex	.agents/skills/ ou .codex/skills/	~/.codex/skills/ ou ~/.agents/skills/
Antigravity	.agents/skills/ (legado: .agent/skills/)	~/.gemini/antigravity/skills/

Tabela 5.2: Onde cada ferramenta espera encontrar skills.

Observe que, em todas as três, não existe mágica, não existe compilação, não existe registro em nenhum servidor. Criou o diretório no caminho certo, escreveu o SKILL.md, a **skill** existe. O agente descobre automaticamente todas as skills disponíveis ao iniciar uma sessão, lendo o diretório correspondente. Se você apagar o diretório, a habilidade desaparece. Se você copiar o diretório para outro projeto, a habilidade viaja junto. É portabilidade total, como um pendrive de habilidades que você espeta em qualquer agente. A boa notícia é que, como o formato do SKILL.md é idêntico nas três, uma skill escrita para Claude Code roda em Codex e Antigravity bastando movê-la para o caminho correspondente.

A anatomia do SKILL.md

O SKILL.md é um arquivo Markdown com uma estrutura específica. Ele começa com um bloco de **frontmatter** em YAML, delimitado por três hifens, que contém os metadados da **skill**. Depois do frontmatter, vem o corpo do arquivo em Markdown, que é o prompt completo que o agente recebe quando a habilidade é ativada. Pense no frontmatter como a etiqueta do cartucho que Tank insere na Nabucodonosor: diz o nome do programa, o que ele faz e quando deve ser usado. O corpo é o programa em si.

```
---
name: revisor-de-codigo
description: >
  Revisa codigo Python seguindo PEP 8 e boas praticas.
  Use esta skill quando o usuario disser: 'revisar codigo',
  'code review', 'verificar qualidade', ou qualquer variacao
  sobre analise de qualidade de codigo.
---

# Revisor deCodigo Python

Agente especializado em revisao de codigo Python.

## Trigger

Ative quando o usuario pedir revisao de codigo.

## Regras

1. Verifique conformidade com PEP 8
2. Identifique code smells
3. Sugira refatoracoes com justificativa

## Fluxo de Execucao

### Passo 1: Ler o codigo
Leia o arquivo indicado pelo usuario.

### Passo 2: Analisar
Aplique as regras de revisao.

### Passo 3: Reportar
Gere um relatorio com os problemas encontrados e sugestoes.
```

Esse exemplo simples já revela a anatomia completa. Toda **skill** tem quatro partes fundamentais: o **frontmatter** com os metadados, o **trigger** que define quando ativar, as **regras**

que governam o comportamento e o **fluxo de execução** que determina a sequência de passos. Vamos examinar cada uma.

O frontmatter: a identidade da skill

O **frontmatter** é o bloco YAML no topo do SKILL.md, entre os delimitadores ---. Ele contém dois campos obrigatórios: name e description. O name é o identificador único da **skill**. A description é o campo mais importante de todo o arquivo, porque é nela que o agente decide se deve ou não ativar aquela habilidade.

Quando o agente recebe uma instrução do usuário, ele lê a description de todas as skills disponíveis e compara com o que foi pedido. Se a descrição menciona 'revisar código' e o usuário disse 'faz um code review nesse arquivo', o agente reconhece a correspondência e carrega a **skill** inteira. É por isso que a descrição precisa ser rica em sinônimos e variações: ela é o mecanismo de busca que conecta a intenção do usuário à habilidade correta.

```
---
name: gerador-de-testes
description: >
  Gera testes unitarios em Python usando pytest. Use esta skill
  SEMPRE que o usuario disser: 'criar testes', 'gerar testes',
  'escrever testes', 'test generator', 'unit tests', 'cobertura
  de testes', ou qualquer variacao sobre criacao de testes
  automatizados.
---
```

Note a palavra 'SEMPRE' em maiúsculas na descrição. Essa é uma convenção que reforça para o agente que, diante daqueles gatilhos, esta **skill** deve ser ativada sem hesitação. Não é obrigatória, mas funciona como um sinal forte de prioridade.

O trigger: quando a habilidade acorda

Logo após o frontmatter, o corpo do SKILL.md começa com o título da **skill** e, em seguida, a seção de **trigger**. O trigger é uma descrição textual das situações em que a habilidade deve ser ativada. Ele complementa a description do frontmatter, mas com mais espaço para detalhar cenários.

A diferença entre a description e o trigger é sutil, mas importante. A description é lida pelo sistema para decidir se a **skill** é candidata. O trigger é lido pelo agente já dentro da **skill**, para confirmar que ele está no lugar certo e entender o contexto de ativação. Pense na description como o rótulo na prateleira de cartuchos da Nabucodonosor e no trigger como

a tela de confirmação antes de injetar o programa.

Na prática, muitos desenvolvedores repetem as mesmas frases nos dois campos. Isso não é redundância; é reforço. O agente processa esses dois campos em momentos diferentes do ciclo de ativação, e a repetição garante que a intenção é capturada em ambos os estágios.

O contexto: o que a skill precisa saber antes de agir

Habilidades não operam no vácuo. Antes de executar qualquer ação, uma **skill** bem construída define quais arquivos, configurações ou informações ela precisa ler. Essa é a seção de contexto obrigatório, geralmente chamada de 'Contexto' ou 'Contexto Obrigatório' no corpo do SKILL.md.

Contexto Obrigatorio

Antes de executar QUALQUER ação, leia:

1. '.book-config.json' - tema e estado atual do projeto
2. '.profile.txt' - perfil do especialista
3. O arquivo '.tex' do capítulo - para manter coerencia

Quando o agente lê essa seção, ele sabe que precisa carregar esses arquivos antes de começar a trabalhar. Sem esse contexto, a **skill** operaria às cegas, como Trinity tentando pilotar um helicóptero sem o programa ter sido completamente carregado. O resultado seria imprevisível, inconsistente e potencialmente destrutivo. O contexto é o que transforma uma instrução genérica em uma habilidade calibrada para o projeto específico em que está sendo usada.

Essa seção também pode incluir **variáveis dinâmicas**. As três ferramentas injetam automaticamente informações como o diretório de trabalho atual, o status do Git, a data de hoje e os argumentos que o usuário passou ao invocar a **skill**. A **skill** recebe essas variáveis em tempo real, sem que o desenvolvedor precise fazer nada. É como se o operador da Nabucodonosor, além de carregar o programa de pilotagem, também injetasse as condições meteorológicas atuais, a altitude do prédio e a posição exata de Morpheus.

As regras: o que a skill pode e não pode fazer

Toda habilidade precisa de limites. Sem regras, o agente interpreta a tarefa da forma que julgar melhor, e 'melhor' para uma máquina nem sempre é melhor para o seu projeto. A seção de regras define explicitamente o que a **skill** deve fazer, como deve fazer e, igualmente importante, o que ela não deve fazer.

```
## Regras de Escrita

### Tom
- Imperativo: 'execute', 'observe', 'perceba'
- Nivel 2 de formalidade

### Proibicoes
- NUNCA use 'pra'. Sempre 'para'.
- NUNCA use 'atraves'. Sempre 'por meio de'.
- NUNCA use bullets no texto final. Texto corrido.
```

Perceba que as regras funcionam em dois eixos. O eixo positivo diz o que fazer: usar determinado tom, aplicar certo formato, seguir uma convenção. O eixo negativo diz o que não fazer: palavras proibidas, padrões vetados, comportamentos indesejados. Os dois eixos são igualmente importantes. Um agente sem proibições vai inevitavelmente fazer algo que você não queria. Um agente sem diretrizes positivas vai entregar algo genérico que não serve para o seu projeto. A combinação dos dois é o que produz uma habilidade realmente útil.

As proibições merecem atenção especial. Note o uso de 'NUNCA' em maiúsculas. Assim como o 'SEMPRE' na descrição, o 'NUNCA' nas regras é um sinal forte para o agente. Não é garantia absoluta de obediência, mas aumenta significativamente a probabilidade de que o limite será respeitado. Pense nisso como um Agent Smith particularmente vigilante dentro da própria **skill**: ele está programado para interceptar exatamente aqueles comportamentos antes que aconteçam.

O fluxo de execução: o programa passo a passo

A última peça fundamental é o **fluxo de execução**. Ele define a sequência de passos que o agente deve seguir ao executar a **skill**. Sem um fluxo explícito, o agente decide sozinho a ordem das operações, e isso pode funcionar para tarefas simples, mas falha espetacularmente em tarefas complexas que exigem dependência entre etapas.

```
## Fluxo de Execucao

### Passo 1: Receber Instrucao
O usuario indica qual secao escrever.

### Passo 2: Carregar Contexto
Leia o .tex do capitulo para saber o que ja foi escrito.

### Passo 3: Planejar Conteudo
Defina internamente:
```

1. Qual o objetivo desta seção?
2. Quais conceitos-chave introduzir?
3. Tem código? Se sim, qual?

Passo 4: Escrever

Gere o LaTeX completo da seção.

Passo 5: Revisar

Verifique conformidade com todas as regras.

O fluxo é o roteiro da missão. Assim como o operador da Nabucodonosor traça a rota antes de inserir a tripulação na Matrix, o fluxo traça a rota que o agente vai percorrer antes de começar a produzir resultados. Cada passo depende do anterior: não faz sentido escrever sem antes carregar o contexto, e não faz sentido carregar o contexto sem saber qual seção o usuário quer.

Como o agente carrega a habilidade

O mecanismo de carregamento funciona em três etapas, e essas três etapas são iguais nas três ferramentas. Primeiro, ao iniciar uma sessão, o agente escaneia todos os diretórios de skills disponíveis e lê o **frontmatter** de cada SKILL.md. Nesse momento, ele não carrega o corpo dos arquivos; apenas indexa os nomes e descrições. É como Tank olhando a prateleira de cartuchos sem inserir nenhum.

Segundo, quando o usuário envia uma mensagem, o agente compara a intenção do usuário com as descrições indexadas. Se houver correspondência, ele carrega o corpo completo do SKILL.md correspondente. A **skill** inteira, com todas as suas regras, contexto e fluxo, é injetada no contexto do agente. A partir desse momento, o agente não é mais um assistente genérico; ele é um especialista com aquela habilidade específica.

Terceiro, existe a ativação explícita pelo usuário. Aqui aparece a primeira diferença real entre as ferramentas, porque o gatilho que cada uma reconhece é distinto:

- No **Claude Code**, o usuário digita / seguido do nome da skill — por exemplo, /revisor-de-codigo. É o **slash command**, ligação telefônica direta para a habilidade.
- No **Codex**, há dois caminhos: o comando /skills abre a lista de skills disponíveis, e a marcação \$nome-da-skill no meio da mensagem invoca uma skill específica diretamente. O Codex reconhece o \$ como prefixo de skill, da mesma forma que outras ferramentas reconhecem a barra.
- No **Antigravity**, a ativação explícita acontece por **workflows**, arquivos colocados em .agents/workflows/ que registram comandos próprios na interface. Cada workflow encadeia skills e personas e fica disponível na barra de comandos da IDE.

Apesar das diferenças de gatilho, o efeito final é o mesmo: a **skill** é carregada imediatamente, sem que o agente precise inferir a intenção. É a ligação telefônica da Matrix: direta, inequívoca, instantânea. O resultado desse mecanismo é que um único agente pode ter dezenas de habilidades disponíveis, mas só carrega na memória a que precisa no momento. Não existe sobrecarga, não existe conflito entre skills. Cada habilidade é um módulo isolado que entra e sai do agente conforme a necessidade. Tank não carrega todos os programas na Trinity ao mesmo tempo. Ele carrega o de pilotagem quando ela precisa pilotar e o de combate quando ela precisa lutar. As três ferramentas fazem exatamente a mesma coisa.

5.3 ACORDANDO NO MUNDO REAL: O QUE É UM AGENTE INTELIGENTE?

Neo passa o filme inteiro sendo um chatbot. Alguém fala com ele, ele responde. Morpheus pergunta, Neo responde. Trinity dá uma ordem, Neo obedece. Ele reage. Não age. Até o momento em que algo muda. Neo começa a perceber a Matrix ao redor, a enxergar os padrões, a decidir sozinho o que fazer antes que alguém peça. Ele para de reagir e começa a agir. Nesse momento, Neo deixa de ser um chatbot e vira um **agente inteligente**.

Essa distinção é a mais importante que você precisa entender antes de seguir adiante neste capítulo, porque todo o ecossistema de **Agent Skills** só faz sentido se o programa que recebe as habilidades for um agente de verdade, não um chatbot fantasiado.

O chatbot responde, o agente resolve

Um chatbot funciona em um ciclo simples: recebe uma mensagem, gera uma resposta, espera a próxima mensagem. Não importa quão sofisticada seja a resposta; o ciclo é sempre o mesmo. Você pergunta, ele responde. Você pergunta de novo, ele responde de novo. Cada interação é uma transação isolada. O chatbot não sabe o que aconteceu antes, não percebe o que está ao redor e não decide fazer nada por iniciativa própria. Se você parar de falar com ele, ele para de funcionar. É Neo antes de acordar: alguém fala, ele reage.

Um **agente inteligente** opera em um ciclo completamente diferente. Ele percebe o ambiente, decide o que fazer com base no que percebeu e age para mudar o estado do ambiente. Depois, percebe o ambiente de novo, porque a ação que ele tomou pode ter mudado as coisas, e o ciclo recomeça. Não precisa que alguém fale com ele para que ele funcione. Ele observa, pensa e age. Quando você pede para um agente ‘corrigir os bugs deste projeto’, ele não responde com uma lista de sugestões e espera você aplicar cada uma. Ele lê os arquivos, identifica os problemas, edita o código, roda os testes, verifica se passou e, se não passou, tenta de novo. Ele resolve. O chatbot teria respondido ‘aqui estão os possíveis

bugs' e ficado esperando a próxima pergunta.

O ciclo do agente: perceber, decidir, agir

O que torna um programa um **agente inteligente** é a presença de três capacidades operando em **loop** contínuo. A primeira é **percepção**: o agente lê o ambiente ao redor. No contexto de qualquer uma das três ferramentas que servem de referência ao livro — **Claude Code**, **Codex** ou **Antigravity** —, isso significa ler arquivos, verificar o status do Git, consultar a saída de um comando, examinar logs de erro. A segunda é **decisão**: com base no que percebeu, o agente escolhe a próxima ação. Não executa tudo de uma vez; avalia o estado atual e decide qual passo faz mais sentido agora. A terceira é **ação**: o agente modifica o ambiente. Edita um arquivo, cria um diretório, roda um teste, instala uma dependência. E a mudança que ele causou altera o ambiente, o que dispara uma nova percepção, uma nova decisão e uma nova ação.

Esse ciclo tem um nome na literatura de inteligência artificial: **loop de percepção-ação**. É o mesmo princípio que governa desde um robô aspirador que percebe obstáculos e desvia até um carro autônomo que percebe o trânsito e freia. A diferença é que, no desenvolvimento de software, o 'ambiente' que o agente percebe é o seu projeto: os arquivos, as dependências, os testes, o histórico do Git, as mensagens de erro do compilador. E as 'ações' que ele toma são as mesmas que um desenvolvedor humano tomaria: editar código, criar arquivos, rodar comandos, refatorar funções.

Para que esse ciclo funcione, o agente precisa de **ferramentas**. Perceber sem ferramentas é impossível: como ler um arquivo sem um comando de leitura? Decidir sem contexto é chute: como escolher a próxima ação sem saber o estado atual do projeto? Agir sem permissões é inútil: como editar código sem acesso ao sistema de arquivos? É por isso que as três ferramentas — **Claude Code**, **Codex** e **Antigravity** — entregam ao agente um conjunto de ferramentas nativas com nomes diferentes mas papéis equivalentes: leitura de arquivos, escrita de arquivos, execução de comandos no terminal, busca por padrões no código, navegação na web. Cada ferramenta é um sentido ou um membro do agente. Sem elas, ele seria um cérebro flutuando sem corpo, como Neo preso na cápsula antes de ser desconectado da Matrix.

O agente não é onisciente

Um equívoco comum é achar que o **agente inteligente** sabe tudo sobre o seu projeto desde o início. Não sabe. Quando uma sessão começa, o agente tem acesso ao diretório de trabalho, ao arquivo de instruções do projeto — `CLAUDE.md` no Claude Code, `AGENTS.md` no Codex e no Antigravity — se existir, e às descrições das **skills** disponíveis. Mas ele não

leu todos os seus arquivos, não memorizou a estrutura do projeto e não sabe o que você fez ontem. Ele descobre sob demanda: quando precisa de uma informação, vai buscá-la. Lê o arquivo, roda o comando, consulta o Git.

Isso é intencional, não é uma limitação. Um agente que carregasse todo o projeto na memória de uma vez seria lento, caro e provavelmente confuso, porque a quantidade de informação em um projeto real excede em muito o que qualquer modelo de linguagem consegue processar de uma vez só. As três ferramentas gerenciam isso com uma janela de contexto que funciona como a memória de trabalho do agente: ele mantém na cabeça o que é relevante para a tarefa atual e descarta o que não é. Quando precisa de algo que já saiu da memória, busca de novo.

Essa dinâmica tem uma consequência prática direta para quem escreve **skills**. Se a sua **skill** precisa de informações específicas do projeto para funcionar bem, ela deve declarar explicitamente quais arquivos ler na seção de contexto obrigatório. Não confie na memória do agente; instrua-o a carregar o que precisa a cada ativação. Um piloto que assume que sabe onde está sem olhar os instrumentos vai bater o helicóptero. Um agente que assume que sabe o estado do projeto sem ler os arquivos vai produzir lixo.

O desenvolvedor como operador

A consequência de tudo isso é uma mudança fundamental no papel do desenvolvedor. Quando o programa que executa suas tarefas é um **agente inteligente** com habilidades modulares, o desenvolvedor não é mais quem digita o código; é quem define as habilidades, configura os limites, monitora a execução e intervém quando necessário. É o operador da Nabucodonosor.

Tank não entra na Matrix. Ele fica do lado de fora, assistindo os monitores, carregando programas, abrindo rotas de fuga, alertando a tripulação sobre perigos. Ele não faz o trabalho de Trinity ou Neo; ele faz o trabalho que permite que Trinity e Neo façam o deles. O desenvolvedor que opera um **agente inteligente** faz a mesma coisa: não escreve cada linha de código, mas escreve as **skills** que ensinam o agente a escrever código do jeito certo. Não roda cada teste, mas configura os **hooks** que garantem que o agente rode os testes antes de cada **commit**. Não monitora cada arquivo, mas define as regras que o agente segue para manter o projeto organizado.

Essa mudança não elimina a necessidade de saber programar. Pelo contrário: exige que o desenvolvedor saiba programar bem o suficiente para ensinar um agente a programar. Quem não entende o que é um **design pattern** não consegue escrever uma **skill** que aplique **design patterns**. Quem não sabe o que é um teste unitário não consegue configurar um **hook** que force o agente a rodar testes. O operador precisa entender a missão melhor do que a

tripulação, porque é ele quem define as regras do jogo. A diferença é que agora ele define as regras uma vez e o agente as aplica indefinidamente, em vez de aplicá-las manualmente a cada arquivo, a cada função, a cada **commit**.

5.4 AGENT SMITH ESTÁ DE OLHO: O QUE SÃO HOOKS?

Em toda a trilogia Matrix, existe um personagem que nunca dorme, nunca se distrai e nunca deixa passar nada. Agent Smith. Ele não faz parte da tripulação, não é aliado, não é inimigo que você derrota e ele vai embora. Ele é o sistema de vigilância da Matrix. Aparece quando alguém faz algo que não devia, intercepta a ação e decide o que fazer: bloquear, modificar ou deixar passar com um aviso. Ele não precisa ser chamado. Ele simplesmente está lá, observando cada movimento, esperando o gatilho que o aciona. Nos três ecossistemas que servem de referência ao livro, esse papel é dos **hooks**, com algumas variações na arquitetura: **Claude Code** e **Codex** oferecem hooks de ciclo **PreToolUse** e **PostToolUse** no mesmo modelo conceitual, mudando apenas o formato do arquivo de configuração. **Antigravity** segue uma arquitetura diferente, baseada em *políticas de permissão* (Allow / Deny / Review) configuradas pela IDE, com efeito final equivalente: ações arriscadas são interceptadas e exigem aprovação antes de executar. Os três caminhos são detalhados a seguir.

Um **hook** é um comando de shell que o sistema executa automaticamente quando o agente realiza determinada ação. O agente vai editar um arquivo? O **hook** pode rodar antes para verificar se o arquivo é protegido. O agente acabou de fazer um **commit**? O **hook** pode rodar depois para executar os testes e garantir que nada quebrou. O agente vai instalar uma dependência? O **hook** pode interceptar e pedir confirmação. O desenvolvedor não precisa ficar vigiando cada ação do agente em tempo real; ele configura os **hooks** uma vez e eles vigiam por ele. É delegar a vigilância para um Agent Smith que trabalha a seu favor.

Dois momentos, dois Smiths

Os **hooks** operam em dois momentos distintos do ciclo de ação do agente. O primeiro é o **PreToolUse**: o **hook** é executado antes que o agente use uma ferramenta. O segundo é o **PostToolUse**: o **hook** é executado depois que a ferramenta terminou de rodar. Pense no **PreToolUse** como o Agent Smith que aparece na porta antes de você entrar e pergunta 'tem certeza de que quer fazer isso?' E no **PostToolUse** como o Agent Smith que aparece na saída e inspeciona o que você fez antes de deixar você seguir em frente.

A diferença prática é crucial. Um **hook** de **PreToolUse** pode impedir a ação. Se o comando retornar um código de saída diferente de zero, o agente não executa a ferramenta.

O **hook** de **PostToolUse** não impede nada, porque a ação já aconteceu; ele serve para reagir ao resultado, registrar o que foi feito ou disparar uma ação complementar.

Onde os hooks moram

Os **hooks** não ficam dentro das **skills**. Eles vivem no arquivo de configuração da ferramenta. Essa separação é intencional: **skills** definem o que o agente sabe fazer; **hooks** definem o que acontece quando ele faz. São responsabilidades diferentes e moram em lugares diferentes. O caminho exato muda entre as três ferramentas:

- **Claude Code**: `.claude/settings.json`, chave `hooks`. Formato JSON.
- **Codex**: `~/.codex/config.toml` (global do usuário) ou `.codex/config.toml` (do projeto, carregado apenas se o projeto for confiável); alternativamente arquivos `hooks.json` ao lado das camadas de `config`. Formato TOML para hooks inline. O Codex aceita hooks de ciclo `PreToolUse` e `PostToolUse` com a mesma semântica do Claude Code, complementados pelas *políticas de aprovação* (`approval_policy`) e *modo de sandbox* (`sandbox_mode`) que limitam o universo de ações possíveis antes mesmo dos hooks rodarem.
- **Antigravity**: configurações da IDE em “Customizations”, com listas Allow / Deny / Review por ação, e `AGENTS.md` do projeto para regras gerais. Não há hooks de ciclo do mesmo formato; o controle vem das políticas de permissão.

A seguir, o mesmo hook — bloquear edição de arquivos protegidos — aparece nos formatos do **Claude Code** e do **Codex**.

```
// Claude Code: .claude/settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "command": "python scripts/check_protected.py \"$FILE_PATH\""
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "command": "python scripts/log_action.py \"$TOOL_NAME\" \"$EXIT_CODE\""
      }
    ]
  }
}
```

```
# Codex: ~/.codex/config.toml ou .codex/config.toml
[[hooks.PreToolUse]]
matcher = "^Edit$"

[[hooks.PreToolUse.hooks]]
type = "command"
command = 'python scripts/check_protected.py "$FILE_PATH"'
timeout = 30
statusMessage = "Verificando arquivo protegido"

[[hooks.PostToolUse]]
matcher = "^Bash$"

[[hooks.PostToolUse.hooks]]
type = "command"
command = 'python scripts/log_action.py "$TOOL_NAME" "$EXIT_CODE"'
timeout = 30
```

Observe a estrutura. Em ambos os formatos, cada **hook** tem dois campos essenciais: o **matcher**, que define qual ferramenta dispara o **hook**, e o **command**, que define o comando de shell a ser executado. O **matcher** é um filtro. No Claude Code, é uma string simples como "Edit" ou "Bash"; no Codex, é uma expressão regular como "^Edit\$" ou "^Bash\$". Se o filtro for * (Claude Code) ou ".*" (Codex), o **hook** dispara em qualquer ferramenta. É como configurar o Agent Smith para vigiar apenas portas específicas ou todas as portas do edifício.

No **Antigravity**, o controle acontece em outro nível, sem arquivo de hooks de ciclo. Em vez de declarar comandos de shell para cada ferramenta, o desenvolvedor classifica cada tipo de ação em uma das três listas da IDE: *Allow* (executa sem perguntar), *Deny* (bloqueia direto), *Review* (pausa e pede aprovação do humano). Para ações de terminal, há ainda a *Terminal Execution Policy* e a *Allow List* de comandos pré-aprovados. O efeito prático é o mesmo de um **PreToolUse** que retorna código diferente de zero: a ação não acontece sem autorização.

PreToolUse na prática: o Smith que bloqueia

O cenário mais comum para um **hook** de **PreToolUse** é proteção. Imagine que o seu projeto tem um arquivo de configuração de produção que nunca deve ser editado pelo agente. Sem um **hook**, nada impede o agente de abrir esse arquivo e modificá-lo se ele julgar que faz sentido no contexto da tarefa. Com um **hook** de **PreToolUse**, você intercepta qualquer tentativa de edição e verifica se o arquivo alvo é o protegido.

```
# scripts/check_protected.py
import sys
```

```
PROTECTED = [
    'config/production.yaml',
    '.env',
    'credentials.json'
]

file_path = sys.argv[1]

for protected in PROTECTED:
    if file_path.endswith(protected):
        print(f'BLOQUEADO: {file_path} e protegido.')
        sys.exit(1)

sys.exit(0)
```

O script recebe o caminho do arquivo como argumento, verifica se ele está na lista de arquivos protegidos e, se estiver, imprime uma mensagem e retorna código de saída 1. O **Claude Code** interpreta qualquer código de saída diferente de zero como uma recusa: a ferramenta não é executada, e o agente recebe a mensagem impressa pelo script como feedback. Ele sabe que foi bloqueado, sabe por quê e pode decidir o que fazer a seguir, como pedir ao usuário que edite o arquivo manualmente. O mesmo efeito é alcançado no **Codex** ao listar os arquivos no modo de sandbox como somente leitura, e no **Antigravity** ao deixar que a IDE solicite aprovação manual antes de qualquer escrita em arquivos sensíveis.

Esse padrão é poderoso porque funciona sem que o agente precise saber das regras de antemão. Você não precisa escrever 'nunca edite o arquivo production.yaml' em todas as **skills**. O **hook** impede a ação independentemente de qual **skill** esteja ativa, independentemente do contexto, independentemente da intenção do agente. É uma camada de segurança que opera abaixo do nível das habilidades, como o Agent Smith que intercepta qualquer um na Matrix, não importa quem seja ou o que esteja fazendo.

PostToolUse na prática: o Smith que inspeciona

O **PostToolUse** serve para reagir ao que já aconteceu. O cenário clássico é rodar testes depois que o agente editou um arquivo de código. Se os testes passarem, tudo segue normalmente. Se falharem, o **hook** pode registrar a falha, notificar o desenvolvedor ou simplesmente deixar o feedback disponível para que o agente perceba o problema no próximo ciclo de percepção.

```
# scripts/run_tests_after_edit.py
import subprocess
```

```
import sys

result = subprocess.run(
    ['python', '-m', 'pytest', 'tests/', '-q'],
    capture_output=True,
    text=True
)

if result.returncode != 0:
    print('ALERTA: testes falharam apos a edicao.')
    print(result.stdout)
    print(result.stderr)
else:
    print('Testes passaram.')

sys.exit(0)
```

Note que o script sempre retorna código de saída zero. Diferente do **PreToolUse**, o **PostToolUse** não bloqueia nada, porque a ação já foi executada. Ele serve para gerar informação, não para impedir. A saída do script é injetada de volta no contexto do agente, que agora sabe que os testes falharam e pode decidir corrigir o problema no próximo passo do **loop de percepção-ação**. É o Agent Smith que não te impede de sair do prédio, mas anota exatamente o que você fez lá dentro.

Hooks não são skills

A confusão entre **hooks** e **skills** é comum, mas a distinção é fundamental. Uma **skill** ensina o agente a fazer algo novo. Um **hook** monitora o que o agente faz, independentemente de qual **skill** esteja ativa. A **skill** é o programa de pilotagem; o **hook** é o Agent Smith que verifica se o helicóptero está autorizado a decolar. A **skill** opera dentro do agente; o **hook** opera fora dele, no nível do sistema.

Essa separação existe por um motivo de segurança. Se as regras de proteção estivessem dentro das **skills**, o agente poderia, em teoria, ignorá-las ou reinterpretá-las de acordo com o contexto. Um **hook** no Claude Code, uma política de sandbox no Codex ou uma exigência de aprovação no Antigravity são executados pelo runtime da ferramenta, não pelo agente. O agente não tem poder para desativar, modificar ou contornar nenhum dos três. São barreiras estruturais, não sugestões comportamentais. Agent Smith não pede licença para aparecer. Ele simplesmente aparece.

Na prática, um projeto bem configurado combina os dois. As **skills** definem as habilidades: como escrever código, como gerar documentação, como criar testes. Os **hooks** definem as restrições: não editar arquivos protegidos, rodar testes depois de cada edição, bloquear

comandos destrutivos no terminal. Juntos, eles formam um ambiente onde o agente é poderoso o suficiente para ser útil e limitado o suficiente para ser confiável. Trinity com o programa de pilotagem e Agent Smith vigiando cada manobra. É assim que se opera a Matrix sem perder o controle.

5.5 O CONSTRUCT: AMBIENTES ISOLADOS COM .VENV

No primeiro filme da trilogia, existe um lugar que não é a Matrix e não é o mundo real. É o Construct: um espaço vazio, branco, infinito, onde a tripulação carrega armas, treina combate e testa simulações sem nenhum risco de afetar o mundo lá fora. Se Trinity testa uma manobra no Construct e explode o helicóptero, ninguém morre. Se Neo leva um soco do Morpheus no Construct, ele acorda sem hematoma. O Construct é um ambiente isolado por definição: o que acontece dentro dele fica dentro dele.

No desenvolvimento de software com Python, esse ambiente isolado tem nome: **.venv**. E quando você cria **skills** que dependem de bibliotecas Python, o **.venv** não é um luxo; é uma necessidade estrutural que separa um projeto profissional de uma bomba-relógio.

O problema que o isolamento resolve

Imagine que você tem duas **skills** no mesmo projeto. A primeira gera gráficos usando `matplotlib` na versão 3.8. A segunda processa dados com `pandas` na versão 2.0, que por acaso exige `matplotlib` na versão 3.9. Se as duas **skills** compartilham o mesmo ambiente Python, você tem um conflito: instalar a versão que uma precisa quebra a outra. Esse problema tem um nome clássico na computação: **dependency hell**, o inferno das dependências.

Sem isolamento, cada biblioteca instalada para uma **skill** afeta todas as outras. É como se cada programa carregado na Trinity alterasse o sistema operacional dela permanentemente. O programa de pilotagem instala os reflexos de aviação, mas como efeito colateral sobrescreve os reflexos de combate corpo a corpo. Na próxima luta, Trinity tenta desviar de um soco e em vez disso faz uma curva de aproximação para pouso. O resultado é desastroso, e a causa é invisível: ninguém percebe que o conflito existe até que algo quebra em produção.

O **.venv** resolve isso criando um Construct para cada contexto. Cada ambiente virtual tem sua própria cópia do interpretador Python e suas próprias bibliotecas, completamente separadas do sistema e de qualquer outro **.venv**. O que você instala dentro de um **.venv** não existe fora dele. O que existe fora dele não interfere no que está dentro.

Criando o Construct

Criar um **.venv** é um comando. Ativá-lo é outro. Os dois juntos levam menos de cinco segundos.

```
# Criar o ambiente virtual
python -m venv .venv

# Ativar no Linux/macOS
source .venv/bin/activate

# Ativar no Windows
.venv\Scripts\activate
```

Depois de ativado, qualquer pacote instalado com `pip` vai para dentro do **.venv**, não para o Python global do sistema. Quando você desativa o ambiente com o comando `deactivate`, volta ao Python global como se nada tivesse acontecido. É entrar e sair do Construct: lá dentro, as regras são suas; lá fora, o mundo continua intacto.

O .venv dentro de uma skill

Quando uma **skill** precisa de dependências Python, a prática recomendada é criar o **.venv** dentro do diretório do projeto, não dentro do diretório da **skill**. O motivo é simples: a **skill** é um conjunto de instruções que o agente segue, e essas instruções devem dizer ao agente como preparar o ambiente do projeto. O **.venv** pertence ao projeto, não à habilidade.

```
meu-projeto/
  .claude/
    skills/
      gerador-graficos/
        SKILL.md          # instrucoes da skill
      .venv/              # ambiente virtual do projeto
      requirements.txt    # dependencias do projeto
      src/
        main.py
```

Observe que o **.venv** fica na raiz do projeto, no mesmo nível que o `requirements.txt`. A **skill** referencia esse ambiente nas suas instruções, dizendo ao agente para ativá-lo antes de executar qualquer script Python. Isso garante que, quando o agente rodar um código que depende de `matplotlib` ou `pandas`, ele estará usando as versões corretas, instaladas no ambiente isolado.

Ensinando a skill a usar o Construct

Para que o agente saiba que precisa ativar o **.venv**, a **skill** deve incluir essa instrução no fluxo de execução. Não é automático; o agente não adivinha que existe um ambiente virtual no projeto. Você precisa dizer a ele, assim como Tank precisa dizer à Trinity qual programa carregar.

```
## Fluxo de Execucao

### Passo 1: Preparar o ambiente
Verifique se o '.venv' existe na raiz do projeto. Se nao existir,
crie com 'python -m venv .venv'. Em seguida, ative o ambiente e
instale as dependencias do 'requirements.txt':

'''
source .venv/bin/activate
pip install -r requirements.txt
'''

### Passo 2: Executar o script
Com o ambiente ativado, execute o script Python da tarefa.

### Passo 3: Verificar o resultado
Confirme que a saida foi gerada corretamente.
```

Perceba que o fluxo começa verificando se o **.venv** existe. Essa verificação é importante porque o agente pode estar sendo usado pela primeira vez no projeto, quando o ambiente ainda não foi criado, ou em uma máquina diferente onde o **.venv** precisa ser recriado. O comando `pip install -r requirements.txt` garante que todas as dependências estejam instaladas e nas versões corretas, mesmo que o **.venv** tenha acabado de ser criado do zero.

O requirements.txt: a lista de armamentos

O arquivo `requirements.txt` é a lista de tudo que o **.venv** precisa conter. Pense nele como o inventário de armas que Tank carrega no Construct antes de uma missão: cada item está listado com a quantidade exata.

```
# requirements.txt
matplotlib==3.8.5
pandas==2.2.2
numpy==1.26.4
requests==2.31.0
```


Os números após o `==` são as versões exatas. Isso não é preciosismo; é proteção. Se você deixar sem versão, o `pip` instala a mais recente, que pode ser diferente da que você testou. Uma atualização silenciosa de biblioteca já quebrou projetos inteiros. Fixar versões é garantir que o Construct de hoje seja idêntico ao Construct de amanhã. É reprodutibilidade, um dos pilares da engenharia de qualquer coisa que funcione.

Para gerar o `requirements.txt` a partir de um ambiente que já está funcionando, basta rodar `pip freeze > requirements.txt` com o `.venv` ativado. O comando captura todas as bibliotecas instaladas com suas versões exatas e grava no arquivo. Na próxima vez que alguém, humano ou agente, precisar recriar o ambiente, basta rodar `pip install -r requirements.txt` e o Construct estará pronto, idêntico ao original.

Por que o agente precisa disso

Quando qualquer uma das três ferramentas — **Claude Code**, **Codex** ou **Antigravity** — executa uma **skill** que roda código Python, ela usa o interpretador Python que encontrar no PATH do sistema. Se não houver um `.venv` ativado, será o Python global, que pode não ter as bibliotecas necessárias ou, pior, pode ter versões incompatíveis. O agente não vai reclamar antes de tentar; ele vai tentar e falhar, e a mensagem de erro vai ser um `ModuleNotFoundError` ou um `ImportError` críptico que consome tempo para diagnosticar.

Com o `.venv` configurado e a **skill** instruída a ativá-lo, o agente opera dentro de um ambiente controlado onde tudo que ele precisa está disponível e nada que ele não precisa interfere. É o Construct: seguro, previsível, isolado. Quando a missão termina, o agente desativa o ambiente e o sistema global continua exatamente como estava. Nenhum efeito colateral, nenhuma surpresa, nenhuma biblioteca fantasma que aparece meses depois quebrando outro projeto.

A combinação de **skills** bem escritas, **hooks** (ou políticas equivalentes) que vigiam e `.venv` que isola é o que transforma **Claude Code**, **Codex** e **Antigravity** de assistentes que respondem perguntas em agentes que operam dentro de um ecossistema controlado. Trinity com as habilidades certas, Agent Smith vigiando cada ação e o Construct garantindo que nenhum experimento contamine o mundo real. Falta uma peça: juntar tudo em uma missão prática. É o que faremos na próxima seção.

5.6 PROJETO PRÁTICO: A MISSÃO

Toda teoria deste capítulo converge para este momento. Você aprendeu o que são **Agent Skills**, como funcionam por dentro, o que é um **agente inteligente**, como os **hooks** vigiam

cada ação e como o **.venv** isola dependências. Agora é hora de juntar todas as peças em uma missão completa: criar uma **skill** que, ao ser invocada em **Claude Code**, **Codex** ou **Antigravity**, estrutura automaticamente um novo projeto Python, prepara o ambiente virtual, instala as dependências e roda um script que valida a cadeia inteira. É a missão de resgate do Morpheus, mas em vez de helicóptero, o que a Trinity vai pilotar é um projeto Python do início ao fim, em qualquer das três ferramentas.

Nota sobre nomes: *Antigravity*, ao longo deste capítulo e do livro inteiro, refere-se à IDE com agente do Google. Existe também um easter egg homônimo do Python, acessado por `import antigravity`, que abre o navegador em uma tirinha do xkcd. Para evitar ambiguidade, não vamos usar esse easter egg como prova prática desta seção. Em vez dele, vamos usar outro easter egg igualmente famoso do Python, o `import this`, que imprime o *Zen of Python* no terminal.

O Zen como prova de funcionamento

O `import this` é um dos easter eggs mais conhecidos do Python. Quando você executa essa linha em um script ou no *REPL*, o interpretador imprime o *Zen of Python* de Tim Peters, um manifesto em vinte aforismos que orientam a estética da linguagem. É inútil em produção e perfeito para um projeto prático: simples o bastante para não desviar o foco do que importa, que é a construção da **skill**, e visível o bastante para confirmar que tudo funcionou, porque o resultado aparece em letras claras no terminal.

Se o Zen apareceu no terminal, a missão foi um sucesso. Se não apareceu, algo na cadeia falhou: a **skill** não carregou, o **.venv** não foi ativado, a dependência não foi instalada ou o script não foi executado. É um teste binário, sem ambiguidade.

A estrutura do projeto

Antes de escrever a **skill**, defina a estrutura que ela vai criar. Um projeto Python bem organizado segue uma convenção simples: um diretório raiz com o nome do projeto, dentro dele um diretório `src/` para o código, um `requirements.txt` para as dependências, um `.gitignore` para proteger o repositório e o **.venv** para isolar o ambiente.

```
zen-project/
  .venv/           # ambiente virtual (criado pela skill)
  .gitignore       # ignora o .venv e arquivos temporários
  requirements.txt  # dependências do projeto
  src/
    main.py        # ponto de entrada do app
```

Essa estrutura é mínima de propósito. O objetivo não é criar o projeto mais completo do mundo; é criar um projeto funcional por meio de uma **skill** que o agente executa com autonomia. Cada arquivo tem uma razão de existir e nenhum arquivo existe sem razão.

Escrevendo o SKILL.md

Agora vem a parte central: o SKILL.md da **skill** criadora de projetos. Ele precisa conter o **frontmatter** com nome e descrição, o trigger, o contexto, as regras e o fluxo de execução completo. Observe com atenção, porque este é o primeiro SKILL.md que você escreve do zero.

```
---
name: criar-projeto-python
description: >
  Cria a estrutura completa de um novo projeto Python com
  ambiente virtual, dependências e app inicial. Use esta skill
  SEMPRE que o usuário disser: 'criar projeto', 'novo projeto',
  'iniciar projeto python', 'project creator', ou qualquer
  variação sobre criar um novo projeto Python do zero.
---

# Criador de Projeto Python

Skill que estrutura um novo projeto Python com .venv,
dependências instaladas e um app funcional.

## Trigger

Ative quando o usuário pedir para criar um novo projeto Python
ou iniciar um projeto do zero.

## Regras

1. O nome do projeto é informado pelo usuário como argumento.
   Se não for informado, use 'meu-projeto'.
2. NUNCA crie o projeto dentro de um diretório que já exista.
   Verifique antes.
3. O .venv DEVE ser criado e ativado antes de instalar
   dependências.
4. O .gitignore DEVE incluir .venv/ e __pycache__/.
5. O requirements.txt começa vazio ou com dependências mínimas.
6. O arquivo main.py DEVE ser executável e funcional.

## Fluxo de Execução
```

```

### Passo 1: Criar a estrutura de diretorios
Crie o diretorio raiz com o nome do projeto e o subdiretorio
src/ dentro dele.

### Passo 2: Criar o .gitignore
Crie o arquivo .gitignore com as entradas padrao para Python.

### Passo 3: Criar o requirements.txt
Crie o arquivo requirements.txt. Se o projeto tiver
dependencias especificas, liste-as aqui.

### Passo 4: Criar o ambiente virtual
Execute 'python -m venv .venv' dentro do diretorio do projeto.

### Passo 5: Ativar o ambiente e instalar dependencias
Ative o .venv e execute 'pip install -r requirements.txt'.

### Passo 6: Criar o main.py
Crie o arquivo src/main.py com o codigo do app.

### Passo 7: Executar o app
Com o .venv ativado, execute 'python src/main.py' para
confirmar que tudo funciona.

### Passo 8: Inicializar o Git
Execute 'git init' e faca o primeiro commit com a mensagem
'feat: estrutura inicial do projeto'.

```

Leia esse SKILL.md com atenção. Ele tem tudo que discutimos nas seções anteriores. O **frontmatter** com nome e descrição rica em sinônimos. O trigger que confirma quando ativar. As regras que definem limites claros, incluindo a verificação de segurança no passo 1. E o fluxo de execução com oito passos sequenciais, cada um dependendo do anterior. O agente que recebe essa **skill** sabe exatamente o que fazer, em que ordem e o que não fazer.

Instalando a skill na ferramenta certa

Para que o agente reconheça a **skill**, ela precisa estar no caminho que a sua ferramenta procura. O conteúdo do SKILL.md é idêntico para as três; muda apenas o diretório de instalação. Use o comando correspondente à ferramenta que você usa no dia a dia.

```

# Claude Code
mkdir -p .claude/skills/criar-projeto-python

# Codex (OpenAI) - escolha um dos dois

```

```
mkdir -p .agents/skills/criar-projeto-python
# ou
mkdir -p .codex/skills/criar-projeto-python

# Antigravity (Google)
mkdir -p .agents/skills/criar-projeto-python
```

Depois, crie o arquivo `SKILL.md` dentro do diretório recém criado, com o conteúdo que escrevemos na subseção anterior. Você pode fazer isso manualmente em qualquer editor de texto ou, se já estiver usando uma das três ferramentas, pedir ao próprio agente que crie o arquivo. A partir do momento em que o `SKILL.md` existir dentro do diretório, a **skill** está disponível.

Note que Codex e Antigravity compartilham exatamente o mesmo caminho (`.agents/skills/`). Uma skill instalada para uma das duas é visível para a outra automaticamente. O Claude Code mantém o seu caminho próprio (`.claude/skills/`), então se você usa as três ferramentas no mesmo repositório, é comum criar duas pastas e manter o `SKILL.md` sincronizado entre elas, ou usar um *symlink* de uma para a outra.

Invocando a skill

Com a **skill** instalada, abra a sua ferramenta e invoque a habilidade. Há duas formas em todas as três: *linguagem natural*, deixando o agente reconhecer a intenção pela descrição, e *invocação explícita*, com o gatilho específico de cada ferramenta. A linguagem natural é igual nas três: escreva “cria um novo projeto Python chamado zen-project” e pronto. A invocação explícita muda:

```
# Claude Code: slash command
/criar-projeto-python zen-project

# Codex (OpenAI): prefixo $ ou comando /skills
$criar-projeto-python zen-project

# Antigravity (Google): workflow registrado em .agents/workflows/
# (apos registrar, aparece como comando proprio na barra da IDE)
```

Em qualquer das três, o agente vai executar os oito passos do fluxo: criar os diretórios, escrever o `.gitignore`, gerar o `requirements.txt`, criar o **.venv**, ativar o ambiente, instalar as dependências, criar o `main.py` e executar o script. Se tudo correr bem, o Zen of Python aparece no terminal e você tem um projeto Python completo, versionado com Git, com ambiente virtual isolado, criado por um **agente inteligente** que seguiu uma **skill** que você

mesmo escreveu.

O main.py do Zen

O código do `main.py` é deliberadamente simples. O objetivo é verificar que toda a cadeia funcionou, não construir um aplicativo complexo.

```
# src/main.py
import this

print()
print('Projeto criado com sucesso!')
print('Se o Zen of Python apareceu acima, a skill funcionou.')
```

Quando o agente executa esse script com o `.venv` ativado, o `import this` dispara a impressão do Zen of Python no terminal. As mensagens no final confirmam que o script rodou até o último `print`. É simples, é verificável e é o suficiente para validar que cada peça do ecossistema está no lugar: a **skill** foi carregada, o fluxo foi seguido, o ambiente foi preparado e o código foi executado.

O que você acabou de construir

Pare um momento e perceba o que aconteceu. Você não criou um projeto Python. Você criou uma **habilidade** que cria projetos Python. A diferença é monumental. O projeto é um artefato; a habilidade é uma capacidade reutilizável. Na próxima vez que você precisar de um novo projeto Python, não vai repetir os mesmos comandos, não vai copiar arquivos de um projeto antigo, não vai lembrar qual versão do `.gitignore` usar. Vai invocar a **skill** e ela faz tudo, toda vez, do mesmo jeito, sem erro, sem esquecimento.

Isso é o que separa o operador do digitador. O digitador cria um projeto por vez, manualmente, repetindo passos toda semana. O operador cria uma **skill** uma vez e o agente cria projetos por ele indefinidamente. Tank não pilota o helicóptero; ele carrega o programa de pilotagem na Trinity e ela pilota. Você não cria o projeto; você carrega a **skill** no agente e ele cria. E se amanhã o padrão de projeto mudar, você atualiza o `SKILL.md` e todas as próximas execuções já seguem o novo padrão. Sem retreinamento, sem reconfiguração, sem e-mail para a equipe explicando a mudança. É a Matrix funcionando a seu favor.

Este capítulo começou com Trinity precisando pilotar um helicóptero e recebendo a habilidade em dois segundos. Ao longo das seções, você entendeu o que são **Agent Skills**, como funcionam por dentro, o que torna um programa um **agente inteligente** de verdade, como os **hooks** (e os mecanismos equivalentes de aprovação e sandbox) vigiam cada ação

do agente e como o **.venv** garante isolamento entre ambientes. Tudo isso vale igualmente para **Claude Code**, **Codex** e **Antigravity**. Agora, com a missão prática concluída, você tem não só o conhecimento teórico, mas uma **skill** funcional, portátil entre as três ferramentas, que prova que cada conceito opera na prática. Nos próximos capítulos, esse ecossistema será a base sobre a qual construímos software de verdade, com **requisitos**, **TDD**, especificações e **deploy**. O operador da Nabucodonosor está pronto. A tripulação aguarda ordens.

CAPÍTULO 6

SDD e TDD: a Mentalidade

Numa orquestra sinfônica, oitenta músicos sentam diante de seus instrumentos, abrem a partitura na primeira página e tocam Beethoven em uníssono. Nenhum deles improvisa. Nenhum deles pergunta o que vem depois. Nenhum deles inventa uma frase melódica achando que vai dar certo. A partitura já disse tudo: qual nota, em que compasso, em que dinâmica, em qual andamento. O regente não toca instrumento nenhum. Ele coordena a execução de algo que foi escrito por outra pessoa, em outro século, e que continua sendo executado fielmente porque alguém, algumas vezes séculos antes, teve o trabalho meticuloso de escrever a partitura.

Agora imagine substituir esses oitenta músicos por oitenta agentes de inteligência artificial. Eles também não vão improvisar, se você der a partitura certa. Eles também não vão perguntar o que vem depois, se a partitura disser o que vem depois. Eles também vão produzir uma sinfonia, e não uma jam session caótica, se houver uma fonte da verdade escrita, precisa, modular e executável. Essa partitura, no desenvolvimento moderno de software, tem um nome: **spec**. E a abordagem que coloca a spec no centro do projeto, em vez do código, chama-se **Spec-Driven Development**, ou SDD.

No Capítulo 1 falamos sobre vibe coding, a prática de pedir código para a IA sem processo, sem contexto e sem regras. Vibe coding é o oposto exato da orquestra. É a jam session do bar: divertida, espontânea, imprevisível e impossível de reproduzir. No Capítulo 4 vimos que manutenção representa dois terços do custo total de um software, e que sistemas mal estruturados acumulam dívida até ficarem impossíveis de mexer. No Capítulo 5 conhecemos as Agent Skills, módulos de habilidade que transformam um agente genérico em um colaborador especializado. Tudo isso converge para o tema deste capítulo: como dar aos agentes uma partitura que eles possam ler, executar e validar, sem que cada trecho do código vire uma surpresa.

Este capítulo tem duas partes complementares. A primeira é o **Spec-Driven Development**: a disciplina de escrever a especificação antes do código, e de tratar essa especificação como o ativo mais valioso do projeto. A segunda é o **Test-Driven Development**, o TDD, que